TS*

언어 메커니즘과 프로그래밍 표현

TypeScript·JavaScript 기능이 제품 요구를 담는 방식

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

독자
JS/TS 중급 이상

사용 방식
언어 해부 · 표현 선택

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

문법이 아니라 요구와 연결한다

이 코드베이스에서 언어 기능은 취향이 아니라 시간·상태·경계·안전 요구를 압축한 표현이다.

제품 요구	언어 메커니즘	대표 표현	얻는 것
점진적 결과	async iterator	`async function*`, `yield`	한 이벤트씩 관찰
단계 합성	generator delegation	`yield*`	하위 stream의 투명한 전달
유효 상태	discriminated union	`type` 필드 + narrowing	분기 안전성
취소 전파	AbortSignal	signal 전달·aborted 검사	협력적 중단
능력 계약	generic + schema	`Tool<Input,Output,Progress>`	정적/런타임 경계
optional feature	dynamic import + gate	`feature()`, `import()`	번들 선택·지연 로딩
자원 정리	explicit resource management	`using`	모든 종료 경로 cleanup

표현의 실제 분포

빈도는 중요도를 증명하지 않지만 어떤 메커니즘이 국소 특수기능이고 어떤 것이 코드 전반의 언어인지 보여 준다.

20,744	2,953	50	65
함수형 노드	**async 함수**	**async generator**	**for-await**
body가 있는 function-like	14.2%	generator 55개 중	비동기 소비

5,238	2,443	165	31
await 표현	**type alias**	**satisfies**	**using 선언**
비동기 작업 밀도	interface 83개 대비	literal 정밀도 보존	자원 수명

type alias가 interface보다 압도적으로 많은 것은 union·mapped/conditional type·literal vocabulary를 적극 사용하는 스타일과 연결된다.

await와 yield가 한 함수에 공존하는 이유

생산자는 네트워크·도구를 기다리고, 소비자는 도착한 이벤트를 즉시 관찰해야 한다. Promise 하나로는 두 시간 축을 동시에 표현하기 어렵다.

학습용 축약 의사코드

```
async function* turn(input, signal) {
  const stream = await openModelStream(input, signal)
  for await (const event of stream) {
    yield event                 // 관찰 가능 시점
  }
  const results = await runRequestedTools(signal)
  for (const result of results) yield result
}
```

AWAIT

생산자의 시간

다음 값이 준비될 때까지 함수 내부 진행을 멈춘다.

YIELD

소비자의 시간

현재 값을 밖으로 내보내되 함수 상태는 보존한다.

NEXT()

pull 제어

소비자가 다음 값을 요구할 때 생산이 다시 진행된다.

RETURN/THROW

종료 의미

정상 terminal과 오류 전파를 iterator protocol에 담는다.

근거 위치 src/query.ts · ECMAScript async generator spec

yield는 여러 번 반환하는 문법이 아니다

yield는 실행 프레임을 보존한 채 iterator result를 내보내는 suspension point다. return은 generator를 terminal 상태로 만든다.

속성	`yield value`	`return terminal`	`throw error`
함수 상태	보존	종료	종료
소비 결과	`{value, done:false}`	`{value, done:true}`	next/rethrow 경로
query 의미	stream/message/progress	terminal reason	실패·중단 전파
UI 영향	즉시 projection 가능	turn 마감	error UI/cleanup

영상과 연결

“함수를 여러 번 나눠 실행한다”는 좋은 직관이다. 더 정확히는 continuation과 local state가 suspension point 사이에 보존되고, 소비자 호출이 진행을 조절한다.

yield*는 stream adapter의 합성 도구

상위 generator가 하위 generator의 모든 값을 다시 포장하지 않고 전달하면서 최종 return 값도 받을 수 있다.

query/queryLoop 관계의 개념 축약

```
async function* query(params) {
  const terminal = yield* queryLoop(params)
  recordCompletedCommands()
  return terminal
}
```

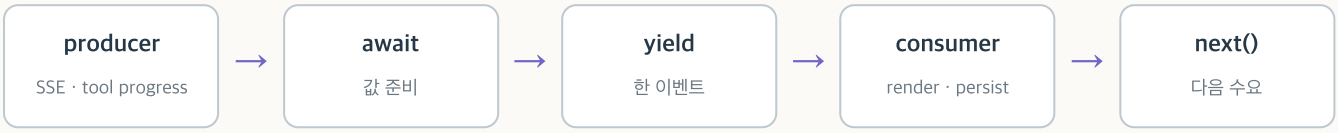
실제 `query()`는 내부 `queryLoop()`의 이벤트를 그대로 노출하면서 정상 종료 뒤 command lifecycle 완료를 기록한다. throw나 iterator의 `.return()`에서는 이 꼬리 코드가 실행되지 않는다는 주석도 있다.

> **설계 포인트**
>
> delegation은 transport wrapping처럼 보이지만 terminal semantics를 바꿀 수 있다. 하위 generator의 throw/return/consumer cancellation이 상위 cleanup과 어떻게 합성되는지 반드시 본다.

근거 위치 src/query.ts

for await는 속도 조절 계약이다

consumer가 다음 iteration을 요청하기 전까지 generator는 다음 yield 이후로 진행하지 않는다. 이 pull semantics가 메모리 폭증과 UI 과잉 갱신을 줄이는 기본 압력 밸브다.



한계

generator protocol만으로 전체 backpressure가 보장되지는 않는다. provider SDK 내부 buffering, React state batching, tool subprocess stdout buffering을 별도로 측정해야 한다.

근거 위치 src/query.ts · src/services/tools/toolOrchestration.ts

제너레이터와 명시적 상태는 경쟁자가 아니다

generator가 instruction pointer와 local frame을 보존하지만, 제품 수준의 재시도·압축·budget·terminal reason은 명시적 데이터 상태로 남겨야 한다.

상태 종류	언어가 보존	코드가 명시
제어 위치	마지막 yield/await 이후 continuation	transition reason
지역 변수	generator frame	messages, turnCount, recovery count
외부 자원	자동 아님	AbortController, prefetch handle, stream executor
재개 가능성	같은 iterator 수명 안	세션 저장/복원은 별도
검증 가능성	간접적	`State`와 `Terminal` union으로 직접

근거 위치 누락된 transitions 모듈의 import 지점 · src/query.ts

이벤트를 “가능한 상태의 집합”으로 만든다

공통 literal 필드가 있으면 TypeScript의 control-flow analysis가 분기마다 정확한 타입을 남긴다.

학습용 union 예시

```
type Outcome =
  | { type: "completed"; value: Message }
  | { type: "denied"; reason: string }
  | { type: "aborted"; by: "user" | "timeout" }

function handle(outcome: Outcome) {
  switch (outcome.type) {
    case "completed": return persist(outcome.value)
    case "denied": return explain(outcome.reason)
    case "aborted": return settle(outcome.by)
    default: return assertNever(outcome)
  }
}
```

이 패턴은 message, permission decision, MCP connection state, terminal reason처럼 네트워크·상태기계 성격을 가진 영역에 특히 잘 맞는다.

근거 위치 src/types/permissions.ts · TypeScript narrowing handbook

never는 “도달 불가”를 컴파일러와 합의하는 장치

새 union member가 추가되었는데 switch가 갱신되지 않으면 `never` 대입이 실패한다. 변화 누락을 런타임 전에 드러낸다.

tool execution의 permission decision source 분기에는 default에서 `const _exhaustive: never = reason` 형태가 보인다. 이 표현은 단순 캐스팅이 아니라 “새 이유가 생기면 여기 또한 갱신하라”는 변경 경보이다.

> **좋은 사용처**
> 보안·결제·상태 전이처럼 새 variant를 묵시적으로 기본 처리하면 위험한 영역. UI cosmetic variant처럼 fallback이 의도된 영역에는 무조건 강제하지 않는다.

근거 위치 src/services/tools/toolExecution.ts · TS exhaustiveness

Tool는 capability의 타입식

한 도구는 입력 스키마, 실행 결과, 진행 이벤트, 권한·동시성·UI 표현을 하나의 계약으로 묶는다.

실제 계약을 축약한 학습 모델

```
type Tool<InputSchema, Output, Progress> = {
  inputSchema: InputSchema
  call(input, context, permission, parent, onProgress): Promise<Result<Output>>
  isConcurrencySafe(input): boolean
  isReadOnly(input): boolean
  checkPermissions(input, context): Promise<Decision>
  renderToolUseMessage(partialInput, options): ReactNode
}
```

> **구조적 긴장**
>
> runtime capability 계약이 ReactNode 렌더링까지 포함해 UI와 core를 결합한다. vertical cohesion에는 유리하지만 headless reuse와 dependency cycle에는 비용이 있다.

근거 위치 src/Tool.ts

buildTool은 런타임 spread를 타입 수준에 복제한다

default method를 채우되, 호출자가 제공한 구체 literal·함수 시그니처는 잃지 않도록 mapped/conditional type을 사용한다.

BuiltTool의 핵심 모양

```
type Built<D> = Omit<D, DefaultKeys> & {
  [K in DefaultKeys]-?:
    K extends keyof D
      ? undefined extends D[K] ? Defaults[K] : D[K]
      : Defaults[K]
}
```

이 표현은 object spread의 “뒤 값이 앞 값을 덮는다”는 런타임 규칙을 타입 계산으로 옮긴다. `-?`는 결과에서 optional을 제거한다.

> **검토 포인트**
>
> `as BuiltTool<D>` 단언은 구현과 타입 계산이 어긋나도 컴파일러가 자동 증명하지 못한다. default 목록과 runtime spread를 함께 테스트해야 한다.

근거 위치 src/Tool.ts

검증하면서 구체성을 잃지 않기

`satisfies`는 값이 계약을 만족하는지 검사하지만, 타입 annotation처럼 표현식의 구체 타입을 넓히지 않는다.

ANNOTATION

타입을 지정

안전하지만 property literal·tuple 정보가 넓어질 수 있다.

ASSERTION

타입을 강제

검증을 우회할 수 있어 경계에서는 위험하다.

SATISFIES

호환성만 검사

literal inference를 유지하며 typo/누락을 잡는다.

PROJECT

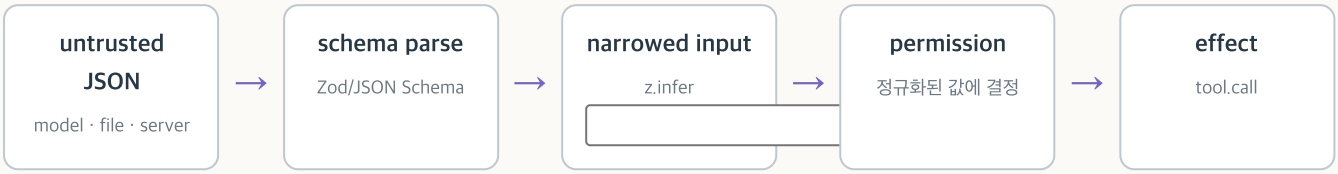
165회 사용

config·registry·tool 정의처럼 literal vocabulary가 중요한 곳에 어울린다.

근거 위치 TypeScript 4.9 satisfies

타입은 네트워크 경계를 지키지 못한다

모델이 생성한 tool input, 설정 파일, MCP payload는 런타임 값이다. Zod/JSON Schema가 정적 타입과 실제 데이터 사이를 잇는다.



소스에는 Zod import가 128개 파일에 나타난다. Tool 계약은 `inputSchema`와 optional `outputSchema`, MCP용 JSON Schema를 함께 수용한다.

> **순서가 중요**
> 권한 규칙을 raw input에 적용한 뒤 값을 정규화하면 검사와 실행 대상이 달라질 수 있다. parse·path canonicalization·permission·effect의 순서를 추적한다.

근거 위치 src/Tool.ts · MCP tool schema spec

AbortSignal은 취소의 공통 통화

취소는 스레드를 강제 종료하는 것이 아니라, 관찰자들이 같은 signal을 확인하고 스스로 정리하는 협력적 프로토콜이다.

단계	signal 사용	놓치면 생기는 문제
model request	SDK/fetch에 signal 전달	사용자 ESC 뒤에도 토큰·비용 소비
tool execution	시작 전/진행 중 aborted 확인	중단 뒤 파일·shell 효과 계속
subprocess	kill/cleanup과 결합	orphan process
prefetch	dispose/ignore late result	session leak
UI	reason별 상태 복원	spinner·input lock 잔류

> **한계**
> AbortSignal은 rollback이 아니다. 이미 발생한 외부 효과는 별도 보상·settlement가 필요하다.

근거 위치 src/Tool.ts · src/query.ts · Node AbortController

React hook은 state 수명과 closure 시점을 표현한다

596개의 useState, 353개의 useRef, 322개의 useEffect, 293개의 useCallback 호출이 interactive surface의 상태 밀도를 보여 준다.

표현	주 역할	대표 위험
useState	render에 반영되는 값	과도한 state 분할·update storm
useRef	render 사이 mutable identity	UI가 ref 변화에 자동 반응하지 않음
useEffect	외부 subscription/lifecycle	cleanup 누락·dependency 오류
useCallback	child/consumer에 안정된 함수 identity	stale closure
useMemo	파생값·object identity	잘못된 성능 가정
useSyncExternalStore	외부 store와 React 일관성	snapshot 계약 위반

REPL 소스의 많은 주석이 “한 렌더 늦은 값”, ref reset 시점, onQuery의 finally와 같은 closure/lifecycle 문제를 직접 기록한다.

근거 위치 src/screens/REPL.tsx

object spread는 변경 경계를 보이게 한다

query loop는 여러 mutable 변수를 흩어 갱신하는 대신 다음 iteration 직전에 `state = { ... }`로 전이 스냅샷을 만든다.

개념 상태 전이

```
state = {
  ...previousState,
  messages: nextMessages,
  turnCount: previousState.turnCount + 1,
  transition: { reason: "next_turn" },
}
```

장점은 diff·테스트·logging에 유리한 전이 경계를 얻는 것이다. 단점은 큰 객체 복사와 nested mutable value(Map, Set, controller)가 "완전 불변"처럼 보일 수 있다는 점이다.

근거 위치 src/query.ts

동적 import와 compile-time 선택

optional subsystem은 번들에 포함되지 않을 수 있고, 어떤 경로는 gate가 true일 때만 require/import 된다.

COMPILE TIME

feature()

빌드 variant에서 dead branch를 제거하도록 설계된 intrinsic.

RUNTIME

import()

필요한 시점에 모듈을 로드하고 startup/cycle 비용을 늦춘다.

TYPE ONLY

typeof import

실행 로딩 없이 optional module의 shape를 참조한다.

RISK

환경별 미검증

활성 조합마다 type/runtime coverage가 달라질 수 있다.

근거 위치 src/query.ts · QUICKSTART.md

using은 generator의 모든 종료 경로를 닫는다

generator는 정상 return뿐 아니라 throw, consumer의 `.return()`, 취소로 끝난다. explicit resource management는 scope 종료 시 dispose를 호출하도록 한다.

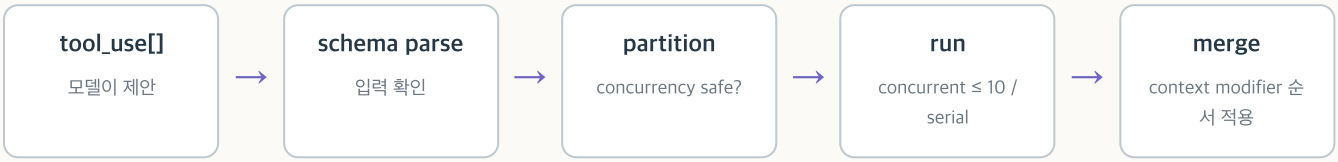
query loop는 memory prefetch handle을 `using`으로 묶어 조기 종료에서도 정리되도록 한다. 소스 전체에서 using 선언은 31개다.

> **generator와 궁합**
> yield로 함수 수명이 길어질수록 manual finally는 누락되기 쉽다. using/await using은 자원 소유권을 lexical scope에 붙인다.

근거 위치 src/query.ts · TC39 Explicit Resource Management

Promise 병렬성과 효과 안전성은 별개다

도구 호출은 `isConcurrencySafe(input)`로 묶음을 나누고, 안전한 연속 블록만 제한된 동시성으로 실행한다.



> **핵심 구분**
> read-only는 concurrency-safe의 충분조건이 아니다. 같은 캐시·rate limit·session state를 갱신하는 읽기 도구도 직렬화가 필요할 수 있다.

근거 위치 src/services/tools/toolOrchestration.ts · src/Tool.ts

오류는 throw 하나로 끝나지 않는다

agent runtime에는 user-visible error message, retryable provider error, tool result error, permission denial, abort, terminal reason이 서로 다른 의미를 가진다.

표현	의미	다음 행동
throw	현재 stack/iterator 실패	상위 recovery 또는 turn failure
assistant API error message	모델 호출 실패를 대화 event로 projection	withhold/recover 또는 표시
tool_result is_error	모델이 다음 reasoning에서 볼 실행 실패	후속 model call 가능
permission deny	효과 불허	설명·수정·중단
AbortSignal	취소 요청	cooperative cleanup
Terminal union	loop 종료 이유	caller settlement

근거 위치 Terminal/Continue import 지점 · src/services/tools/toolExecution.ts

언어 기능을 작은 실험으로 검증하기

대형 소스를 읽기 전 60줄 이하의 toy program으로 semantics를 손에 익힌다.

1. **generator** — three-event async generator를 만들고 `.next()`, `.return()`, throw를 직접 호출한다.
2. **backpressure** — producer 10ms, consumer 100ms 지연을 주고 메모리·순서를 기록한다.
3. **abort** — model mock과 tool mock에 같은 signal을 전달하고 어느 단계에서 멈추는지 표로 만든다.
4. **union** — terminal variant 하나를 추가해 exhaustiveness error를 확인한다.
5. **generic builder** — default method를 채우면서 literal name 타입을 보존하는 builder를 구현한다.
6. **runtime schema** — raw JSON을 parse하기 전/후 permission check 결과가 달라지는 반례를 만든다.

실습 · 축약 query loop

모델 mock이 text→tool_use를 내고 tool mock이 결과를 반환한 뒤 두 번째 모델 호출에서 completed가 되는 루프를 작성한다.

완료 증거: event 순서, terminal reason, abort test

언어 선택의 핵심 결론

좋은 코드는 고급 문법을 많이 쓰는 코드가 아니라, 시간·상태·경계 요구를 가장 작은 정확한 표현으로 만든 코드다.

시간

async generator

점진 결과와 내부 대기를 하나의 composable protocol로 만든다.

상태

union + never

불가능한 상태와 변경 누락을 줄인다.

경계

generic + schema

정적 계약과 untrusted runtime 값을 이어 준다.

제어

AbortSignal + using

취소와 cleanup을 구조화한다.

다음 권

03권에서는 이 표현들이 실제 query loop 안에서 어떻게 압축·모델·tool·회복·종료 상태기계로 합쳐지는지 본다.